

Leva.Framework.Sample.WebIdentity

```
1  using System.Security.Claims;
2  using Leva.Framework.Core;
3  using Leva.Framework.Identity;
4  using Leva.Framework.Identity.AspNet;
5  using Leva.Framework.Identity.Local;
6  using Leva.Framework.Identity.Memory;
7  using Leva.Framework.Sample.WebIdentity;
8  using FrameworkAuthenticationService = Leva.Framework.Identity.AuthenticationService;
9  using FrameworkAuthorizationService = Leva.Framework.Identity.AuthorizationService;
10 using FrameworkSystemClock = Leva.Framework.Core.SystemClock;
11
12 var builder = WebApplication.CreateBuilder(args);
13
14 // Samples may run as Production; load user secrets explicitly for local Google OAuth.
15 builder.Configuration.AddUserSecrets<Program>(optional: true);
16
17 var principalStore = new MemoryPrincipalStore();
18 var sessionStore = new MemoryAuthSessionStore();
19 var sessionService = new AuthSessionService(sessionStore);
20 var localServices = LocalPrincipalServices.Create(principalStore);
21 var authorization = new FrameworkAuthorizationService([new BuiltInAuthorizationPolicy()]);
22 var apiPermission = new PrincipalPermission("api.use");
23
24 var admin = new Principal(
25     PrincipalId.New(),
26     "JWT Admin",
27     "jwt-admin@example.test",
28     Roles: new HashSet<PrincipalRole> { new("admin") },
29     Permissions: new HashSet<PrincipalPermission> { apiPermission }
30 );
31
32 principalStore.Add(admin, "admin");
33 await localServices.CredentialService.CreateAsync("admin", "password", admin.Id);
34
35 // Google can be configured through user secrets, .NET environment variables, or explicit environment
36 // variables.
37 var googleClientId =
38     builder.Configuration["Google:ClientId"] ??
39     Environment.GetEnvironmentVariable("LEVA_GOOGLE_CLIENT_ID");
40 var googleClientSecret =
41     builder.Configuration["Google:ClientSecret"] ??
42     Environment.GetEnvironmentVariable("LEVA_GOOGLE_CLIENT_SECRET");
43 var googleConfigured = !string.IsNullOrEmpty(googleClientId) &&
44     !string.IsNullOrEmpty(googleClientSecret);
45
46 builder.Services.AddSingleton<IClock, FrameworkSystemClock>();
47 builder.Services.AddSingleton<IPrincipalStore>(principalStore);
48 builder.Services.AddSingleton<ILocalCredentialStore>(localServices.Credentials);
49 builder.Services.AddSingleton(localServices.CredentialService);
50 builder.Services.AddSingleton<IAuthSessionStore>(sessionStore);
51 builder.Services.AddSingleton(sessionService);
52 builder.Services.AddSingleton(authorization);
53
54 // Compose framework authentication policies for the ASP.NET adapters.
55 builder.Services.AddSingleton<AuthenticationPolicy>(localServices.AuthenticationPolicy);
56 builder.Services.AddSingleton<FrameworkAuthenticationService>(provider => new FrameworkAuthenticationService(
57     provider.GetServices<AuthenticationPolicy>().ToArray(),
58     sessionService
59 ));
60
61 // Cookie/session identity lets the Google callback sign the user into the browser.
62 builder.Services.AddAspNetPrincipalServices(options =>
63 {
64     options.AllowHeaderSession = true;
65     options.SecureCookie = false;
66 });
```

```

62 });
63
64 // The JWT provider issues and validates bearer tokens from framework auth sessions.
65 builder.Services.AddAspNetJwtPrincipalServices(options =>
66 {
67     options.Issuer = "Leva.Framework.Sample.WebIdentity";
68     options.Audience = "Leva.Framework.Sample.WebIdentity";
69     options.SigningKey = "sample-development-signing-key-32-bytes-minimum";
70     options.TokenLifetime = TimeSpan.FromMinutes(30);
71 });
72
73 if (googleConfigured)
74 {
75     builder.Services.AddAspNetGooglePrincipalServices(options =>
76     {
77         options.ClientId = googleClientId!;
78         options.ClientSecret = googleClientSecret!;
79         options.SecureCookie = false;
80         options.DefaultReturnUrl = "/";
81     });
82 }
83
84 builder.Services.AddAuthorization(options =>
85 {
86     options.AddPolicy(
87         "JwtApiUse",
88         policy =>
89         {
90             // Require JWT authentication here, not the cookie handler.
91             policy.AuthenticationSchemes.Add(AspNetJwtDefaults.AuthenticationScheme);
92             policy.Requirements.Add(
93                 new
94                 AspNetAuthorizationRequirement(AuthorizationRequirement.Permission(apiPermission))
95                 );
96         });
97 });
98
99 var app = builder.Build();
100
101 app.UseDefaultFiles();
102 app.UseStaticFiles(
103     new StaticFileOptions
104     {
105         OnPrepareResponse = context =>
106         {
107             // Samples change often while developing, so disable browser caching for local static
108             files.
109             context.Context.Response.Headers.CacheControl = "no-store, no-cache, must-
110             revalidate";
111             context.Context.Response.Headers.Pragma = "no-cache";
112             context.Context.Response.Headers.Expires = "0";
113         },
114     });
115
116 app.UseAuthentication();
117 app.UseAuthorization();
118
119 app.MapGet("/favicon.ico", () => Results.NoContent());
120 app.MapPost(
121     "/sample/logout",
122     (HttpContext context) =>
123     {
124         context.Response.Cookies.Delete(AspNetDefaults.SessionCookieName);
125         return Results.Ok(new { cleared = true });
126     });
127

```

```

128 // The built-in JWT endpoint authenticates local credentials and returns a bearer token.
129 app.MapAspNetJwtEndpoints();
130
131 if (googleConfigured)
132     // Real Google login needs a configured OAuth client and registered callback URL.
133     app.MapAspNetGoogleEndpoints();
134 else
135 {
136     app.MapGet(
137         "/identity/google/login",
138         () => Results.BadRequest(new { message = "Google auth is not configured for this sample
host." })
139     );
140     app.MapGet(
141         "/identity/google/callback",
142         () => Results.BadRequest(new { message = "Google auth is not configured for this sample
host." })
143     );
144 }
145
146 app.MapGet(
147     "/sample/google/status",
148     (HttpContext context) =>
149     {
150         var callbackUrl = $"
{context.Request.Scheme}://{context.Request.Host}/identity/google/callback";
151         return new WebIdentityGoogleStatus(
152             googleConfigured,
153             "/identity/google/login",
154             callbackUrl,
155             googleConfigured
156             ? "Google auth is configured. The login button will start the real OAuth
flow."
157             : "Google auth is not configured. Set Google:ClientId and Google:ClientSecret
with user secrets, or set LEVA_GOOGLE_CLIENT_ID and LEVA_GOOGLE_CLIENT_SECRET."
158         );
159     }
160 );
161
162 app.MapGet(
163     "/sample/google/principal",
164     (HttpContext context) =>
165     {
166         var roles = context.User.FindAll(ClaimTypes.Role).Select(claim => claim.Value).ToArray();
167         var permissions = context
168             .User.FindAll(AspNetClaimsPrincipalMapper.PermissionClaimType)
169             .Select(claim => claim.Value)
170             .ToArray();
171         var name = context.User.Identity?.Name ?? context.User.FindFirst("name")?.Value;
172         var email = context.User.FindFirst(ClaimTypes.Email)?.Value ??
context.User.FindFirst("email")?.Value;
173
174         return new WebIdentityGooglePrincipal(
175             context.User.Identity?.IsAuthenticated == true,
176             name,
177             email,
178             roles,
179             permissions
180         );
181     }
182 );
183
184 app.MapGet(
185     "/api/jwt/admin-area",
186     (HttpContext context) =>
187     {
188         var roles = context.User.FindAll(ClaimTypes.Role).Select(claim =>
claim.Value).ToArray();
189         var permissions = context

```

```

190         .User.FindAll(AspNetClaimsPrincipalMapper.PermissionClaimType)
191         .Select(claim => claim.Value)
192         .ToArray();
193
194         return new WebIdentityApiResponse(
195             permission.",
196             context.User.Identity?.Name,
197             context.User.Identity?.IsAuthenticated == true,
198             roles,
199             permissions
200         );
201     }
202 }
203 .RequireAuthorization("JwtApiUse");
204
205 app.Lifetime.ApplicationStarted.Register(() =>
206 {
207     Console.WriteLine("WebIdentity sample");
208     Console.WriteLine("Open the printed local URL, run JWT checks, and optionally configure Google
209 OAuth.");
210 });
211 app.Run();
212

```